Try out these comprehensions in an IEx session:

```
$ iex
iex> for i <- [1, 2, 3], do: i * 2
[2, 4, 6]

iex> for i <- 1..6,
         Integer.mod(i, 2) == 0,
         into: %{},
         do: {i, "even!"}

%{2 => "even!", 4 => "even!", 6 => "even!"}
```

The first example is the most basic form of a comprehension and is likely what you'll use most of the time. The second shows features that turn comprehensions into a powerful tool.

The `Integer.mod(i, 2) == 0` clause acts as a filter for the data. Elixir checks the data against this function and only executes the iteration function if the result is truthy. In this case, it's checking if the remainder of `i / 2` is 0, which is true only for even numbers.

The `into:` option puts the result into the given data structure. In our example, a map is returned instead of a list. By default, comprehensions are returned as a list (equivalent to `into: []`), so you only need to type this out if you want the result in something other than a list.

The next example extracts data from the `Conversation` struct that we created earlier. This is presented as a `.exs` file, but type it out in an `IEx` shell.

```
$ iex -S mix
```

elixir_examples/convo.exs

```
alias Examples.Structs.{Conversation, Sms}

from_steve = %Sms{
  from: "Steve", to: "Reader", status: "delivered"
}

from_reader = %Sms{
  from: "Reader", to: "Steve", status: "received", body: "text"
}

other = %Sms{
  from: "+1-222-333-4444", to: "Steve", status: "received"
}

failed_other = Map.put(other, :status, "failed")

convos = [
  Conversation.new([from_steve, from_reader, from_steve]),
  Conversation.new([other, failed_other]),
]
```